

---

# **Automated Support for Debt Capital Markets Activities**

*Final Project for the Data Analytics University Diploma  
at Paris I Panthéon–Sorbonne University*

---



# Table of Contents

## Contents

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>0</b> | <b>Project Overview and Disclaimer</b> | <b>1</b>  |
| 0.1      | Project Context . . . . .              | 1         |
| 0.2      | Project Presentation . . . . .         | 1         |
| 0.3      | The Program . . . . .                  | 1         |
| <b>1</b> | <b>Structuring Desk</b>                | <b>2</b>  |
| 1.1      | Pricer . . . . .                       | 2         |
| 1.1.1    | Vanilla bonds . . . . .                | 2         |
| 1.1.2    | Zero-coupon bonds . . . . .            | 4         |
| 1.1.3    | Convertible Options . . . . .          | 6         |
| 1.1.4    | Sustainability-Linked bonds . . . . .  | 8         |
| 1.2      | Tools . . . . .                        | 9         |
| 1.3      | Glossary and Learn More . . . . .      | 10        |
| <b>2</b> | <b>Intelligence Desk</b>               | <b>11</b> |
| 2.1      | AI Agent . . . . .                     | 11        |
| 2.2      | Latest News and Insights . . . . .     | 13        |
| 2.2.1    | News Aggregator . . . . .              | 13        |
| 2.2.2    | Macroeconomics Dashboard . . . . .     | 14        |
| 2.2.3    | Deal Tracker . . . . .                 | 15        |
|          | <b>Conclusion</b>                      | <b>16</b> |
|          | <b>Bibliography</b>                    | <b>17</b> |

# 0 Project Overview and Disclaimer

## 0.1 Project Context

As a Master in Management student at ESCP Business School, I aspire to build a career in Investment Banking. During my first gap-year internship in Equity Capital Markets at BNP Paribas, I developed a strong interest in Global Capital Markets. I now seek to broaden my expertise by exploring other areas of GCM, particularly Debt Capital Markets (DCM), which are the core focus of this study.

The purpose of this program is to provide an automated and educational support tool for DCM activities. Since I have not yet gained direct professional experience in this field, the tool may contain limitations or areas for improvement. Its main ambition is twofold: first, to demonstrate my genuine interest in DCM, and second, to showcase my coding and data visualization skills acquired during my training at Paris I Panthéon–Sorbonne University.

## 0.2 Project Presentation

The program is designed as a Streamlit application divided into two main sections. The first section, the *Structuring Desk*, includes pricing tools for different types of bonds, specialized utilities tailored to DCM practice, and a glossary to clarify technical terms. The second section, the *Intelligence Desk*, integrates an AI Agent as well as tools for monitoring the debt markets, namely a news aggregator, a dashboard of macroeconomic indicators, and a deal tracker.

For educational purposes, the application also provides concrete examples available for download in PDF format, along with explanatory notes to help readers navigate the specific terminology used in DCM.

## 0.3 The Program

The application has been built using Streamlit, a free and easily deployable framework for interactive data apps. It is coded in Python, enabling a wide range of external libraries and supporting research. The different modules of the program are detailed in the following sections.

Finally, the whole project is accessible through: <https://dcm-workbench.streamlit.app/>

# 1 Structuring Desk

*An automated support tool for bond pricing and other DCM-specific utilities*

## 1.1 Pricer

The Pricer page brings together the main valuation tools for vanilla, zero-coupon, convertible, and sustainability-linked bonds. Its aim is both practical and educational: each module captures bond parameters in a `dataclass`, generates cash flows, and applies explicit pricing formulas—discounting for vanilla and zeroes, a binomial tree for convertibles, and KPI-linked coupons for SLBs. Results are shown through interactive inputs, Altair charts, and key metrics. This consistent structure makes the Pricer a rigorous calculator aligned with market practice and a clear learning tool for DCM.

### 1.1.1 Vanilla bonds

The Vanilla Bond pricer illustrates how the value of a plain fixed-rate bond can be determined through three standard approaches, implemented directly in Python. The module is built around a `BondSpec` class that generates the full schedule of coupons and redemption flows, which are then discounted or inverted depending on the chosen method.

**Yield-to-Maturity method.** The clean price is defined as the present value of all future coupons and principal, discounted at the yield-to-maturity  $y$ :

$$P = \sum_{t=1}^N \frac{CF_t}{(1 + ym)^{mt}}$$

where  $CF_t$  are the cash flows,  $m$  the number of coupon payments per year, and  $N$  the total number of periods. In the code, `price_from_yield()` applies this formula directly, while `yield_from_price()` uses a Newton–Raphson loop that adjusts  $y$  until the model price matches the observed market price.

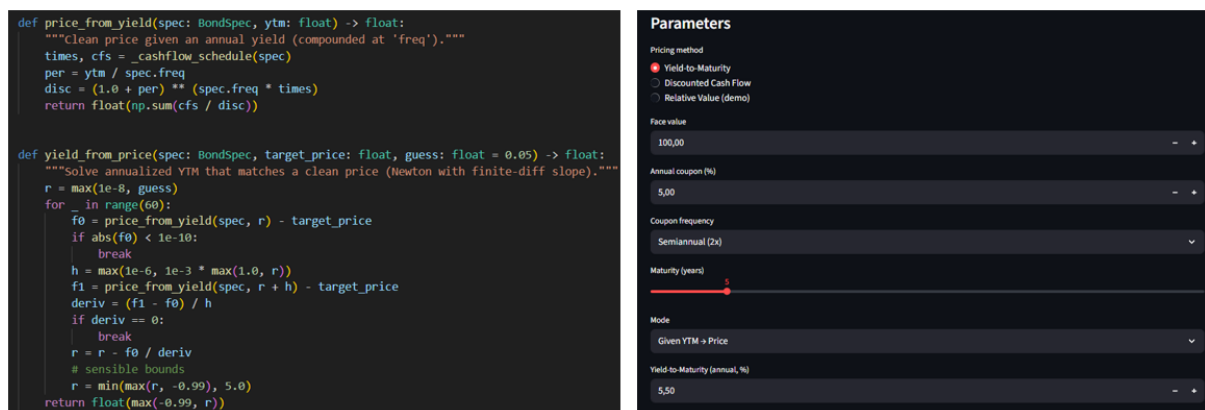


Figure 1: Illustration of the Yield-to-Maturity method.

**Discounted Cash Flow method.** Here the user specifies a discount rate  $r$  instead of solving for the market yield. The formula is identical with  $r$  in place of  $y$ :

$$P = \sum_{t=1}^N \frac{CF_t}{(1 + rm)^{mt}}$$

This shows that YTM pricing is a particular case of discounted cash-flow valuation, linking academic DCF logic and market practice.

**Relative Value method.** The bond is compared with a set of peers. For each peer, the program reconstructs a `BondSpec`, computes its implied yield from the market price using the YTM solver, and collects all yields in a peer table. The median yield  $\tilde{y}$  is taken as a benchmark. The code applies the pricing function with  $\tilde{y}$  to compute the “fair” price of the subject bond and calculates its spread in basis points:

$$Spread(bps) = (y_{our} - \tilde{y}) \times 10,000$$

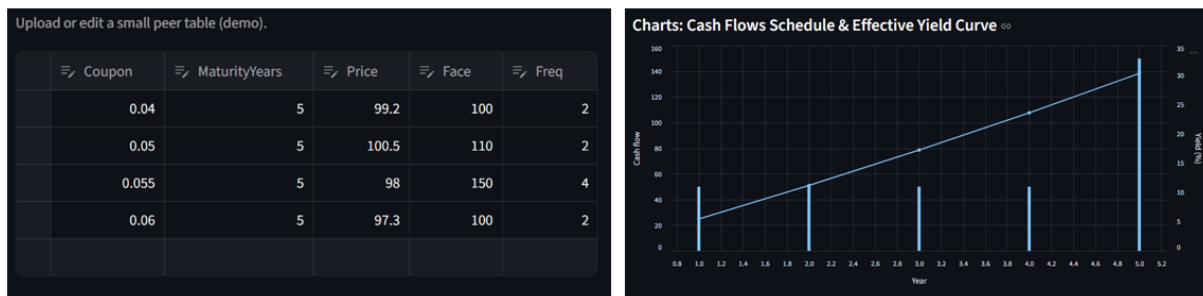


Figure 2: Relative Value: peer table (left) and cash flows and effective-yield chart (right).

On the chart, the bars show the subject bond’s cash flows, while the line plots the effective annual rate at horizon  $t$  when compounding the benchmark YTM, illustrating how discounting intensity increases with maturity rather than forecasting future rates.

**Key metrics.** This section presents key indicators to understand a bond at a glance: *Price* (current value of coupons and redemption), *Yield* (the single annual rate making price match cash flows, like an IRR), *Duration* (average time to get paid back, showing interest rate sensitivity), *Convexity* (refines duration for bigger rate changes), and *Price-to-Par* (price as a percentage of face value).

In the app, these values are calculated from the bond parameters and displayed with Streamlit’s `st.metric` blocks, which show big numbers and labels side by side. A small styling snippet (CSS — the language that controls layout and alignment) centers the text for clarity. Just below, a collapsible panel (`st.expander` “Learn More”) provides short definitions and explanations: users can click to reveal details without cluttering the main screen.

### 1.1.2 Zero-coupon bonds

The Zero-Coupon module focuses on the simplest bond: a single redemption at maturity without interim coupons. The pedagogical aim is to show how the price of this unique cash flow can be obtained either analytically from the spot curve or numerically by simulating the short rate with Monte Carlo methods.

**Analytical (Spot Curve).** We price a zero-coupon by discounting its single payoff at maturity using the market's zero-coupon yield curve (the *spot curve*). This curve gives one rate for each maturity (e.g., 1Y = 3%, 2Y = 3.2%, 5Y = 3.8%). Because the rate changes with maturity, we cannot just apply “3.8% for 5 years” as if growth were constant. The idea is to split the horizon into many small intervals, apply the local rate on each, and multiply all the tiny discount factors. In continuous time, this accumulation becomes an integral of the short rate:

$$P = F \times \exp\left(-\int_0^T r(u) du\right)$$

where the exponential gathers continuous compounding and the minus sign reflects discounting. If the curve is flat (constant  $z$ ), this simplifies to  $P = F e^{-zT}$ .

Here is how we implemented it in our Python code step by step:

1. **Choose the curve mode.** If the user selects a flat zero rate  $z$ , we build  $r(t) \equiv z$  via `_flat_zero_rate_fn`. If the user provides a table of *tenors* and *zero rates*, we create a piecewise-linear function  $r(t)$  with flat extrapolation using `_interp_zero_rate_fn` (tenors sorted; a tiny shift avoids  $t=0$  issues).
2. **Compute the discount factor.** We approximate the integral  $\int_0^T r(u) du$  on a fine grid (with 1,000 steps by default) with the trapezoidal rule in `_continuous_discount_from_curve`, then set

$$df_{cont} = \exp\left(-\int_0^T r(u) du\right)$$

3. **Price.** Multiply by the face value: `price_cont = spec.face * df_cont`.
4. **Display yield (convention).** For reporting, we convert the price into a yield using the selected compounding:

$$y_{cont} = -\frac{\ln(P/F)}{T} \quad \text{or} \quad y_{annual} = \left(\frac{F}{P}\right)^{1/T} - 1$$

as implemented in `analytic_price_zero(..., compounding)`.

5. **Return.** The function returns `(price, display_yield)`.

*Notes.* The interpolated  $r(t)$  is piecewise-linear with flat extrapolation; the integration grid controls accuracy (more steps  $\Rightarrow$  more precise integral). All rates are annual and expressed in decimals.

**Monte Carlo (Short-Rate).** This method prices a zero-coupon bond by simulating many possible future paths for the **short rate**  $r_t$  — the instantaneous risk-free interest rate used to discount cash flows. Instead of assuming one fixed rate, we let  $r_t$  follow a *mean-reverting process*:

$$dr_t = a(\theta - r_t) dt + \sigma dW_t$$

where  $a$  is the speed of mean reversion,  $\theta$  the long-run level the rate tends to return to, and  $\sigma$  the volatility of random shocks. Each simulated path gives its own discount factor  $\exp(-\int_0^T r_s ds)$ . Averaging these discounted payoffs across thousands of simulations produces the bond's fair price, an expected value under uncertain future interest rates.

Here is how we implemented it in our Python code step by step:

1. **Simulate short-rate paths.** We generate  $r_t$  with the Euler–Maruyama scheme for the mean-reverting process  $dr_t = a(\theta - r_t) dt + \sigma dW_t$ . On a grid with step  $\Delta t$ , the update is

$$r_{k+1} = r_k + a(\theta - r_k) \Delta t + \sigma \sqrt{\Delta t} Z_k, \quad Z_k \sim \mathcal{N}(0, 1)$$

This is implemented in `simulate_short_rate_paths()`, which returns the time grid and a matrix of simulated paths. The formula reflects two forces: the term  $a(\theta - r_k)\Delta t$  pulls the rate back toward the long-run average  $\theta$ , while the term  $\sigma\sqrt{\Delta t}Z_k$  adds random shocks scaled to the time step.

*In simple terms: we cut time into many small pieces; at each step the rate drifts back to its average level but is also perturbed by randomness, creating many possible future interest rate paths.*

2. **Price by averaging discounted payoffs.** For each path, we approximate the integral  $\int_0^T r_s ds$  by the trapezoidal rule,

$$\int_0^T r_s ds \approx \sum_{k=0}^{n-1} \frac{1}{2} (r_k + r_{k+1}) \Delta t$$

compute the discount factor  $D = \exp(-\int_0^T r_s ds)$ , and average across paths to get the price  $P = F \cdot E[D]$ . Function `mc_price_zero()` also reports the equivalent continuous yield  $y_{\text{cont}} = -1/T \ln(P/F)$  and keeps a small random sample of paths for plotting. The discount factor comes from continuous compounding: adding up all the tiny interest rates along a path gives the total growth of one euro; taking  $\exp(-\cdot)$  moves back to present value. Averaging across many paths gives the expected price under uncertain rates.

*In simple terms: each simulated rate path tells us how much €1 at maturity is worth today along that scenario. We average these present values to get a fair price, then translate it into a single equivalent yield to compare with market quotes.*

---

|                                                   |                                                          |
|---------------------------------------------------|----------------------------------------------------------|
| $r_t$ instantaneous short rate at time $t$        | $\Delta t$ time step size                                |
| $a$ speed of mean reversion                       | $Z_k$ standard normal random shock at step $k$           |
| $\theta$ long-run average level of the short rate | $F$ face value of the bond                               |
| $\sigma$ volatility of short rate changes         | $P$ present price of the bond                            |
| $W_t$ Brownian motion (source of randomness)      | $y_{\text{cont}}$ equivalent continuous-compounded yield |

---

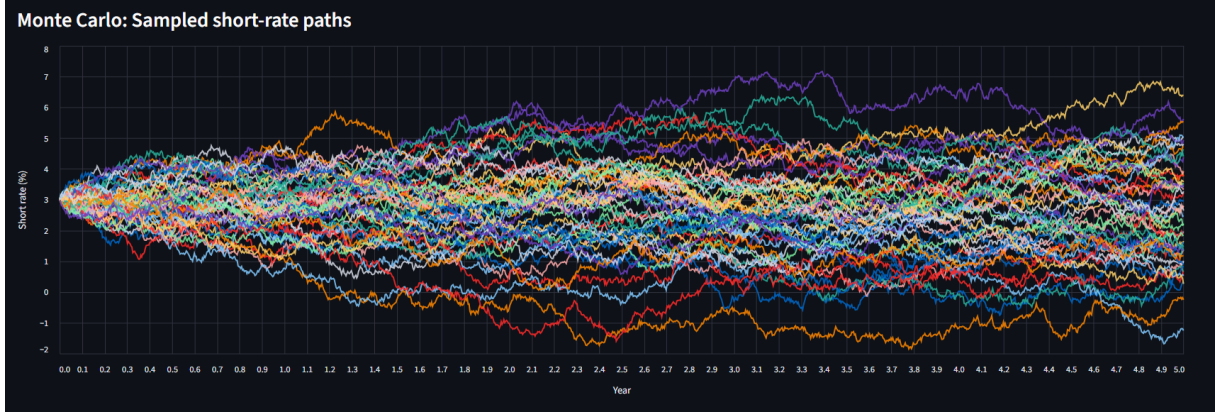


Figure 3: Sampled Monte Carlo short-rate paths. Each line is one possible future of the short rate. The bond price is the average discounted payoff across all these scenarios. More paths reduce randomness, smaller time steps make the approximation more accurate.

### 1.1.3 Convertible Options

This module prices a convertible bond, i.e., a straight bond plus the right to convert into shares, with an optional *issuer call*. The page serves two purposes: (i) compute a fair value aligned with market practice, and (ii) visualize how the *bond floor* protects the downside while the conversion option creates upside convexity and equity-like behaviour when the stock rises. In the app, inputs are grouped in a dataclass `ConvSpec` (face, coupon rate/frequency, maturity,  $S_0$ , dividend  $q$ , volatility  $\sigma$ , risk-free  $r$ , credit spread  $s$ , conversion ratio, call price and start), key metrics are shown via `st.metric`, and the chart overlays model value, bond floor, parity, regime bands, the conversion price (strike) and a conversion premium marker.

**Method — step by step.** We value the security on a Cox–Ross–Rubinstein (CRR) binomial tree under the risk-neutral measure. The implementation mirrors the financial logic in four passes:

1. **Build the equity tree and map coupons.** The equity tree has  $N$  steps of size  $\Delta t = T/N$ ; up/down factors and the risk-neutral probability are set by `_binomial_params`. Coupon dates are translated into step indices (end-of-period payments) by `_times_and_periods`; a boolean vector flags the steps where the fixed coupon (`_coupon_per_period`) is added.
2. **Set the terminal payoff.** At maturity the holder chooses redemption at par or immediate conversion; the code initializes the terminal layer of the tree with this maximum.
3. **Roll back through the tree.** For each node we compute a discounted risk-neutral expectation of next-step values (adding the coupon if due), compare it with immediate conversion, and, if a call is active from the next date, apply the issuer's settlement cap. This backward induction (function `_binomial_convertible_pricer`) returns the fair value at time 0.
4. **Report layers & Greeks.** The straight-bond floor is computed separately by discounting coupons and principal at  $r + s$  with frequency  $m$  (`_straight_bond_price_floor`); parity is  $ratio \cdot S_0$ ; the conversion price (strike) is  $F/ratio$ . Delta and Gamma are obtained by finite differences on  $S_0$  (`_greek_delta_gamma`).



In simple terms, the code builds a lattice of possible future stock prices and, at each step, decides whether the investor would rather continue holding the bond or convert into shares. By rolling these decisions backward through time, the program determines the fair present value of the CB.

Regarding the call option, the code allows the issuer to redeem the bond early once the call date is reached. At each node where the call is active, the model compares the continuation value to the call price: if redeeming is cheaper than letting the bond continue, the issuer is assumed to call. This mechanism limits upside for investors and slightly reduces the bond's value when the stock trades high.

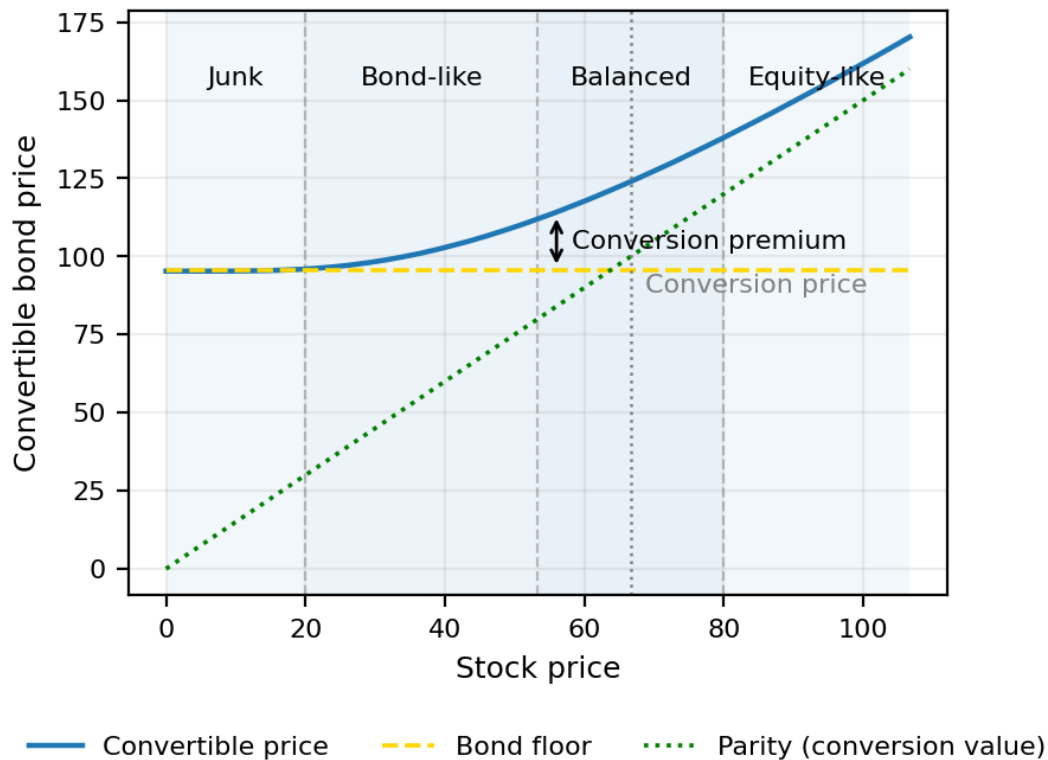


Figure 4: Convertible Bond Price Evolution

The interactive chart plots the convertible bond price (blue) as a function of the underlying stock price. The yellow dashed line shows the bond floor (minimum value if conversion never occurs) while the green dotted line shows the parity (immediate conversion value). Shaded areas represent typical regimes, from *bond-like* to *equity-like*. A vertical dotted line marks the *conversion price*, and a double arrow highlights the *conversion premium*, which is the extra value paid for the embedded option.

The chart helps users see how the convertible behaves across scenarios: steady and protected at low stock prices, increasingly sensitive and convex as the stock rises.

## 1.1.4 Sustainability-Linked bonds

This page presents a case study based on Saint-Gobain's €500m 10-year issuance (Aug. 2022) under its Sustainability-Linked Financing Framework. The goal here is not strict pricing but to show, from the *issuer's* viewpoint, how KPIs can change the final cost of debt.

In this case study, the 2032 coupon may step up depending on two sustainability performance targets (SPTs): a 33% reduction in Scope 1&2 CO<sub>2</sub> emissions and an 80% reduction in non-recovered waste vs. a 2017 baseline. Missing one target triggers a +37.5 bps step-up; missing both triggers +75 bps (only the 2032 payment is affected).



Figure 5: Sustainability-Linked Bond Page - Saint-Gobain Case Study.

The chart shows how the coupon schedule evolves across scenarios. In code, each scenario (*Good*, *Neutral*, *Bad*) is a *Scenario* dataclass with a step-up (bps) and a color. For each year, the program builds the schedule with a base coupon of 2.625% and an adjustment in 2032 that depends on the scenario. Streamlit sliders let users set KPI outcomes (actual reductions in CO<sub>2</sub> and waste). The app compares these inputs to the targets, counts how many KPIs are missed, computes the step-up in basis points, and converts it into euros using the fixed notional (€500m).

The grouped-bar Altair chart renders *Good* / *Neutral* / *Bad* side by side for each year, while the right-hand *Key metrics* panel displays two live indicators: *2032 coupon* and *Expected 2032 incremental cost to the issuer*. Their color updates with the scenario using “traffic-light” codes (green for 0 bps, orange for +37.5 bps, red for +75 bps), implemented via a small CSS-styled `kpi()` helper. This makes SLB mechanics immediately tangible: move the sliders, see the step-up and extra euro cost change in real time.

## 1.2 Tools

The Tools page provides two lightweight, self-contained utilities: a *Term Sheet Builder* to generate a pre-filled Word term sheet from on-screen inputs, and an *Amortization Builder* to compute and visualize repayment schedules.

- **Term Sheet Builder.** Inputs (issuer, currency, status, dates, coupon rate, use of proceeds, etc.) are captured via Streamlit widgets and assembled into a context dictionary whose keys match placeholders in a Word template. The document is produced by `_render_termsheet_docx()` using `docxtpl` (a Python library that fills DOCX templates), then returned to the user with a download button. A guard raises clear errors if `docxtpl` is missing or the template path is invalid. The yield field is computed on the fly by `_ym_from_price()`, which solves the standard bond-pricing equation for Yield to Maturity (YTM) via an iterative Newton-style method: coupons are discounted at a per-period rate  $r$ , updated until present value matches the clean price (the market price excluding accrued interest). Users choose coupon frequency (annual/semi-annual/quarterly), which sets both the discounting periods and the coupon amount per period. This keeps the template consistent with market quoting while avoiding any external pricing service.

|                            |                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------|
| Issuer:                    | {{issuer}}                                                                                                 |
| Guarantor:                 | {{guarantor}}                                                                                              |
| Currency:                  | {{currency}}                                                                                               |
| Rating:                    | {{rating}}                                                                                                 |
| Joint Lead Managers:       | {{joint_lead_managers}}                                                                                    |
| Status:                    | {{status}}                                                                                                 |
| Governing Law:             | {{governing_law}}                                                                                          |
| Issue Amount:              | {{issue_amount}}                                                                                           |
| Trade Date:                | {{trade_date}}                                                                                             |
| Maturity Date:             | {{maturity_date}}                                                                                          |
| Coupon:                    | {{coupon}}                                                                                                 |
| Use of proceeds:           | {{use_of_proceeds}}                                                                                        |
| Issue Date:                | {{issue_date}}                                                                                             |
| Clean Price: (clean_price) |                                                                                                            |
| Accrued: (accrued)         |                                                                                                            |
| Issue Price: (issue_price) |                                                                                                            |
| Issue Yield:               | {{yield}}                                                                                                  |
| Form:                      | Inscribed stock of the issuer issued in registered form by entry in a register maintained by the Registrar |
| Selling Restrictions:      | As set out in the Information Memorandum                                                                   |
| Listing:                   | Yes                                                                                                        |

```

# --- Context for docxtpl (keys must match .docx placeholders) ---
context: Dict[str, Any] = {
    "issuer": issuer.strip(),
    "guarantor": guarantor.strip(),
    "currency": currency.strip() or "EUR",
    "rating": rating.strip(),
    "joint_lead_managers": joint_lead_managers.strip(),
    "status": status,
    "governing_law": governing_law.strip(),
    "governing_law ": governing_law.strip(), # safety for trailing-space tag in Word
    "issue_amount": f"{{currency.strip() or 'EUR'}} {{issue_amount:,.0f}}.replace(", " ")",
    "trade_date": trade_date.strftime("%d %b %Y"),
    "maturity_date": maturity_date.strftime("%d %b %Y"),
    "coupon": coupon_str, # -> {{coupon}}
    "use_of_proceeds": use.strip(),
    "issue_date": issue_date.strftime("%d %b %Y"),
    "clean_price": f"{{clean_price:.2f}}%",
    "accrued": f"{{accrued:.2f}}%",
    "issue_price": f"{{issue_price:.2f}}%",
    "yield": yield_display, # computed
    "today": date.today().strftime("%d %b %Y"),
}

```

Figure 6: Term Sheet Builder — Streamlit Inputs and Context Dictionary for DOCX Generation

- **Amortization Builder.** The function `build_amortization()` generates a schedule as a pandas DataFrame for three structures: *bullet* (interest-only, principal repaid at maturity), *equal principal* (constant principal each period, declining interest), and *annuity* (constant total payment, interest plus principal split varies over time). Parameters are notional, annual rate, maturity in years, and payment frequency; the function returns period index, time in years, beginning balance, interest, principal, total payment, and ending balance. The UI renders the table with formatted columns and an interactive Altair chart that overlays stacked bars (interest vs. principal per period) with a line for outstanding balance; users can zoom/pan and download the schedule as CSV.

Hence, the module favours clarity and robustness: explicit input fields, transparent math, template-driven DOCX generation, and simple exceptions for missing dependencies or files.

## 1.3 Glossary and Learn More

The Glossary & Learn More page is designed as an educational resource within the DCM Workbench. Its goal is to make bond pricing and other metrics easier to understand by providing a clear dictionary of formulas, terms, and interpretations. The glossary is structured for readers who may already know finance but are less familiar with the specific mechanics of Debt Capital Markets. Alongside this, the “Learn More” tab broadens horizons with a non-exhaustive overview of exotic bond types from around the world, encouraging further exploration of DCM instruments. The implementation uses Streamlit tabs to separate two sections: one for the glossary and one

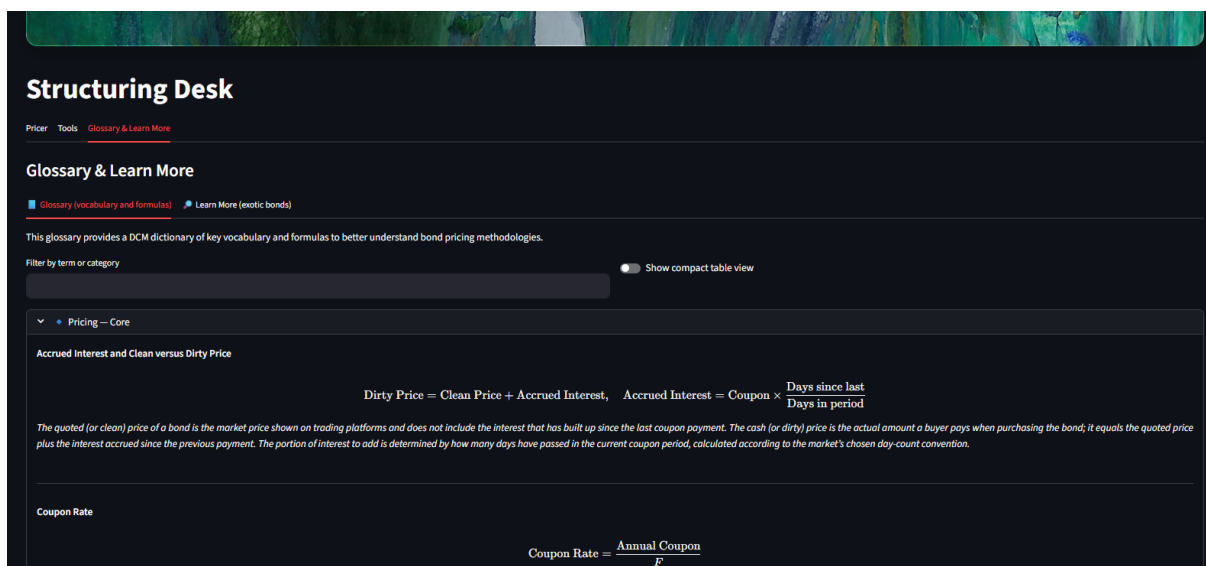


Figure 7: Glossary Page - an overview

for the list of exotic bonds. The glossary content is stored as a DataFrame (a structured table from the pandas library) with fields for category, term,  $\text{\LaTeX}$  formula, and interpretation. The interface includes filters (search by keyword or category) and a toggle between a compact table view and an expanded view with collapsible sections (expanders) for each category. Within each expander, terms are displayed with bold headings, and formulas rendered in  $\text{\LaTeX}$  (mathematical typesetting). A dedicated section at the bottom defines all variables and notation used in the formulas for clarity.

On the “Learn More” side, exotic bond types are grouped into categories (issuer-based, structural, environmental, or next-generation) and displayed as simple bullet points with concise definitions.

## 2 Intelligence Desk

*A tool for AI-powered assistance, document review, and monitoring of financial news and deals*

### 2.1 AI Agent

Building on the machine learning and RAG (Retrieval-Augmented Generation) foundations taught in the Data Analytics program, the application integrates an AI Agent powered by Mistral AI. It answers user queries and supports document-based searches through a drag-and-drop interface that allows users to retrieve specific information directly from uploaded files.

The integration of Mistral AI into the application required a direct connection to its API (Application Programming Interface), the secure channel through which external programs send prompts and receive model responses. This connection was implemented without a dedicated SDK (Software Development Kit), keeping the code transparent and lightweight. Instead, the system uses the `requests` library to send HTTPS requests directly to the endpoint `https://api.mistral.ai/v1/chat/completions`.

Key technical aspects include:

- The API key used for authentication is stored securely in Streamlit's `secrets.toml` or as an environment variable, both excluded from version control via `.gitignore` (not public).
- Model configuration parameters (e.g., `mistral-large-latest`, context length, retry policy) are defined as environment variables for easy modification.
- A system prompt embedded in each request guides the model's tone and logic (concise answers, no invented data, consistent use of document context).
- A retry loop with an exponential back-off mechanism enhances reliability: if the API is temporarily overloaded, the program automatically retries the request with increasing delays (e.g., 1s, 2s, 4s), preventing server congestion and ensuring stable user interaction.

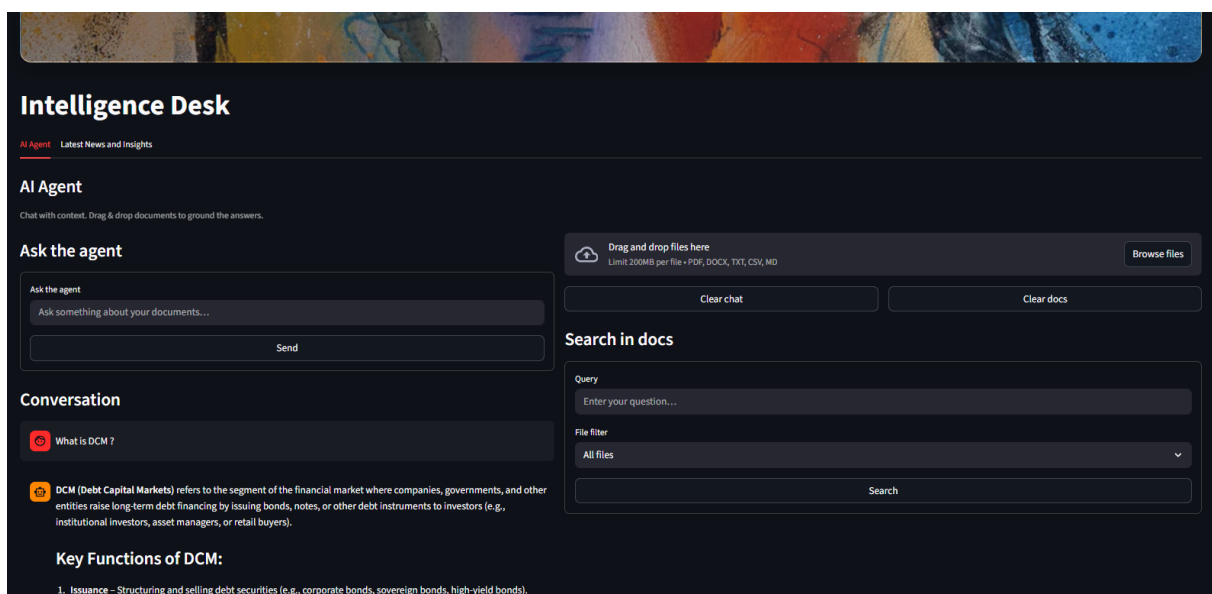


Figure 8: AI Agent page - an overview

When the user submits a query, the application constructs a structured list of messages that serves as the conversational context for the Mistral API. Then, this list is stored in `st.session_state.agent_messages`, includes a predefined system prompt (which defines the assistant's behavior), the most recent dialogue turns (up to six), and any contextual snippets extracted from uploaded documents via `_build_context_from_docs()`. Each element is formatted according to the standard `messages` structure expected by the API, with a defined role ("system," "user," or "assistant") and a corresponding text content.

The complete conversation history is then sent as a JSON payload to the Mistral API, which returns a generated answer displayed in the Streamlit chat interface through `st.chat_message`. In short, the dialogue pipeline follows a clear and minimal sequence:

*Userinput → JSONrequest → Mistralresponse → On – screendisplay.*

This process enables the agent to reason on both previous messages and the uploaded documents.

For document search, `_search_keywords()` scans uploaded files using regular expressions (pattern matching that identifies keywords). Around each match, a short *snippet* is extracted and cleaned from formatting artifacts, then highlighted with HTML tags. These snippets are:

- Displayed directly to the user with contextual highlights;
- Sent to a secondary Mistral query (`_ai_assist_summary()`) that summarizes the information and cites filenames for traceability.

In summary, the AI Agent shows how a compact Python architecture can reproduce the core principles of modern RAG systems. By combining secure communication with the Mistral API, a clear conversational interface, and real-time document search with contextual summarization, it creates an intelligent and interactive workspace for DCM activities.

```
# ----- Config -----
MODEL_NAME = os.getenv("MISTRAL_MODEL", "mistral-large-latest")
MAX_CONTEXT_CHARS = 2000      # total chars from uploaded docs (for chat)
MAX_FILE_CHARS = 4000        # per-file cap to avoid huge payloads
HISTORY_TURNS = 6            # number of prior messages to keep

SYSTEM_PROMPT = """You are a helpful, concise AI assistant embedded in a Debt Capital Markets workspace.
- Always answer clearly and directly.
- Use any provided document context when relevant; if not relevant, ignore it.
- If information is missing, say so briefly rather than inventing facts.
"""

def _ai_assist_summary(query: str, results: Dict[str, List[str]], api_key: str) -> str:
    if not results:
        return ""
    flat = []
    for f, ss in results.items():
        for s in ss[:3]:
            text_only = re.sub("<.*?>", "", s)
            text_only = re.sub(r"\s+", " ", text_only).strip() # normalize for clean summary
            flat.append((f, text_only))
    flat = flat[:5]
    snippet_block = "\n".join([f"- [{f}] {t}" for f, t in flat])
    messages = [
        {"role": "system", "content": "You summarize only using the provided snippets. If unsure, say you cannot find it."},
        {"role": "user", "content": f"Query: {query}\n\nSnippets:\n{snippet_block}\n\nAnswer succinctly and cite filenames in brackets."},
    ]
    return _call_mistral(messages, api_key)
```

Figure 9: Mistral AI configuration and example of the AI Summary function



## 2.2 Latest News and Insights

This page provides an automated solution for monitoring financial markets by centralizing key information (headlines, macroeconomic indicators, and bond issuance data) within a single page.

### 2.2.1 News Aggregator

This page aims to gather and display the latest financial headlines from multiple RSS feeds (“Really Simple Syndication,” a standardized format that automatically distributes updates from news websites), allowing users to filter them by keyword or summarize them automatically with the Mistral AI model. Technically, this module relies on the `feedparser` library, which reads RSS streams from pre-defined sources (*Reuters*, *ECB*, *Federal Reserve*, etc.). The function `fetch_rss()` downloads up to a user-defined number of articles per feed, extracts their titles, publication dates, and links, and merges them into a unified pandas DataFrame sorted by recency. A caching decorator (`@st.cache_data`) stores the results for 15 minutes to reduce redundant network calls.

Once fetched, the headlines are displayed in a dynamic table built by an helper defined as `_render_clickable_table()`, which formats entries in HTML with clickable “Open” links. A keyword filter, based on Python’s `re` (regular expressions, used for pattern matching in text), allows users to focus on relevant terms such as “ECB” or “syndicated.”

Finally, the top headlines can be summarized by the function `summarize_with_mistral()`, which sends them as a text prompt to the Mistral API using an HTTPS request. The model replies with concise English bullet points focused on DCM topics (rates, issuance, and credit markets), demonstrating how a large language model can assist in real-time market monitoring.

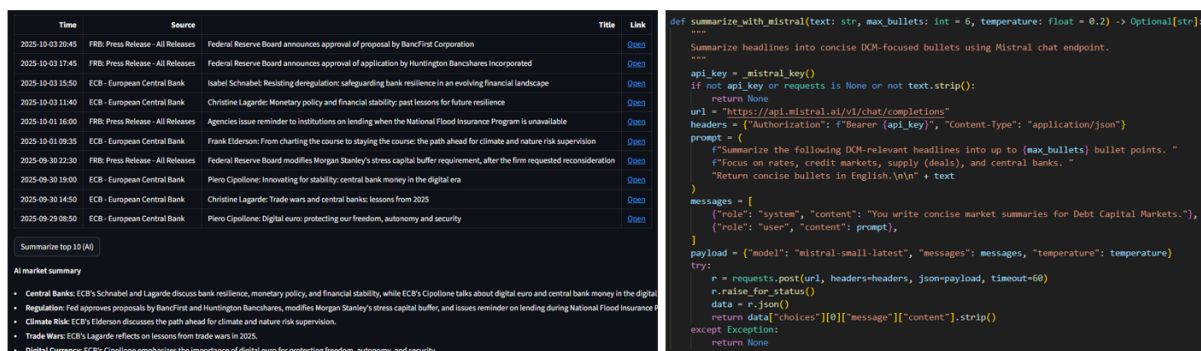


Figure 10: Mistral-powered market summary — Interface and underlying Python function

## 2.2.2 Macroeconomics Dashboard

This dashboard provides a synthetic view of key macroeconomic indicators: central bank policy rates, long-term government bond yields, and yield spreads. Its objective is to illustrate how official data can be combined with financial APIs to visualize interest-rate trends.

The code begins by calling `fetch_policy_rates()`, which retrieves the latest central-bank decisions directly from public sources such as the Federal Reserve’s FRED database (via CSV endpoints) or, for the Bank of England, by scraping the official website. The results are formatted into a table styled with bold values for readability.

Then, the *Latest levels (10Y)* section displays current 10-year government bond yields for the United States, Germany, France, and the United Kingdom, retrieved in real time from `yfinance` (Yahoo Finance’s Python API) and shown via `st.metric` widgets. The function `pct_change()` computes yield variations, while captions show data dates. If live feeds fail, `demo_prices()` generates synthetic data to keep the dashboard functional offline.



Figure 11: Macroeconomics Dashboard - an overview

Finally, the chart *Spreads over the last 10 years*, focuses on long-term yield differentials between key issuers — specifically, the U.S.–France (US–FR) and France–Germany (FR–DE) 10-year government bond spreads. Here, the program loads historical data from local CSV files originally downloaded from the Federal Reserve’s FRED database (<https://fred.stlouisfed.org>). These files, are processed and merged to compute the spreads in percentage points. The resulting time series are plotted with `Altair`, a declarative visualization library, which renders interactive line charts including a zero reference line and tooltips. This visualization highlights how yield spreads have evolved over the past decade, providing insight into the relative market perception of sovereign credit risk among major European and U.S. issuers.



### 2.2.3 Deal Tracker

The Deal Tracker detects and lists recent DCM transactions from press release feeds such as GlobeNewswire, PR Newswire, Euronext, and the London Stock Exchange. Its purpose is to identify new primary market activity and highlight participating banks and issuers.

The page aggregates data from public RSS feeds (Really Simple Syndication). The function `fetch_rss()` collects press-release headlines, returning a table containing the publication time, source, title, and link. The user can adjust the number of headlines per feed or add custom URLs through a text editor field.

The analytical core of this module lies in its use of regular expressions (*regex*), a text-search syntax that allows complex pattern matching. The code defines several patterns to distinguish DCM-related announcements from irrelevant news:

- `_INSTRUMENT` detects bond-related keywords: “*green bond*,” “*notes*,” “*convertible*,” etc.
- `_ACTION` identifies issuance verbs: “*launches*,” “*prices*,” “*mandates*,” or “*bookrunners*.”
- `_NEGATIVE` excludes unrelated topics such as *software*, *earnings*, *IPOs*, etc.

DCM\_PATTERN ensures that only headlines containing both an instrument and an issuance action are kept. This filter is applied by `_is_dcm_headline()`, producing a subset of DCM-relevant titles.

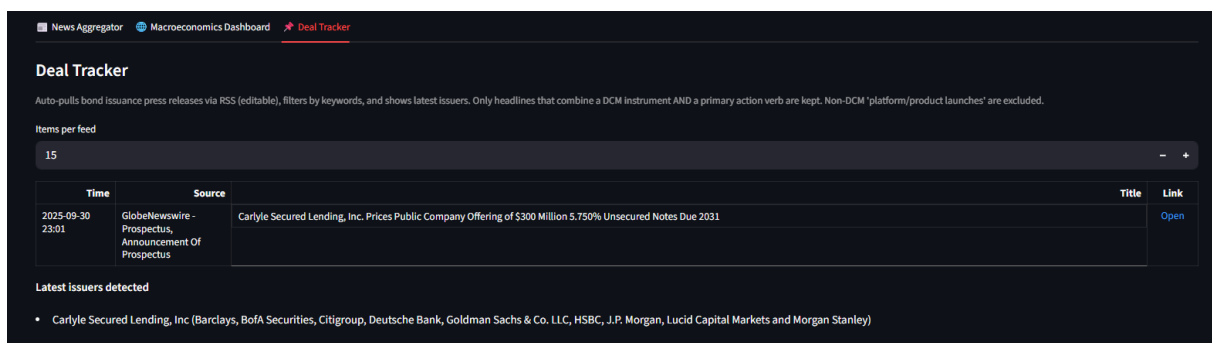


Figure 12: Deal Tracker - an overview

Next, the system extracts key entities. The function `_extract_issuer()` identifies the issuer by parsing text before verbs like “*announces*” or “*issues*” (e.g., “*BNP Paribas launches €500m green bond*” → “*BNP Paribas*”). Meanwhile, `_extract_banks_from_text()` scans the title and body for participating banks using a synonym dictionary and a compiled regex `BANK_PATTERN` to detect name variations (e.g., “*J.P. Morgan*,” “*JP Morgan*,” “*JPMorgan*”).

To enhance accuracy, the app downloads a limited number of article pages (up to 12) and removes all HTML tags, styles, and scripts before text analysis. This cleaning step is performed by `_fetch_article_plaintext()`, which uses the `requests` library to retrieve the full page and the `html.unescape()` function to convert encoded characters into readable text. The extracted text is then analyzed for bank mentions, complementing the information from the headline.

Finally, the results are rendered in a compact, clickable table generated by the function `_render_clickable_table()`, which displays publication time, source, title, and a direct link to the article. The program also compiles a short summary of detected issuers and associated bookrunners, shown as a bullet list under “*Latest issuers detected*.” If no relevant news is found, the interface provides user-friendly feedback messages instead of errors.

## Conclusion

This project set out to design and implement an educational platform bringing together essential tools and resources for Debt Capital Markets. By combining a Structuring Desk dedicated to bond pricing models, specialized utilities, and a glossary of financial terms with an Intelligence Desk focused on AI assistance, news monitoring, and market dashboards, the application serves both as a practical toolkit and a pedagogical companion.

Although not exhaustive and open to future refinements, the platform demonstrates how coding and data visualization can make complex market mechanisms more accessible. It also illustrates the intersection between financial expertise and data-driven analysis, bridging the quantitative methods developed through the Data Analytics program at Paris I Panthéon–Sorbonne with the corporate finance training acquired at ESCP Business School.

Looking ahead, the application could be further enhanced through live data integrations, broader coverage of financial instruments, and advanced AI-driven features. Such developments would reinforce its dual purpose as an educational resource and an operational support tool for market professionals.

# Bibliography

## Structuring Desk

Antona, A. *Bond-Pricing* — GitHub repository. Available at: <https://github.com/Anthony-Antona/Bond-Pricing>.

Bank of England. *GDP-Linked Bonds — London Term Sheet (discussion draft)*, November 2015. Available at: <https://www.bankofengland.co.uk/-/media/boe/files/events/2015/november/gdp-linked-bonds-london-term-sheet-2>.

Bond-Pricing Documentation. *Python library for fixed income valuation*. Available at: <https://bond-pricing.readthedocs.io/en/latest/>.

Bond Theory Reference. *Introduction to Bond Valuation and Pricing. Educational Document*. Available at: <https://www.investopedia.com/terms/b/bond-valuation.asp>.

Djoleglc. *Pricing Convertible Bonds using Cox–Ross–Rubinstein model* — GitHub repository. <https://github.com/djoleglc/Pricing-Convertible-Bonds-using-Cox-Ross-Rubinstein-model>.

HSBC Bank plc. *Issue of USD 50,000,000 Callable Zero Coupon Notes due September 2043. Pricing Supplement*, 25 September 2023. Available at: <https://www.hsbc.com>.

Hoffmann Green Cement Technologies. *Issuance of Bonds Convertible into New Shares and/or Exchangeable for Existing Shares (OCEANE), EUR 5 million nominal amount. Press Release*, 16 February 2024. Available at: <https://www.euronext.com>.

Latham & Watkins LLP. *Demystifying Convertible Bonds — A Practical Primer*, April 2024. Available at: <https://www.lw.com/en/insights-landing/admin/upload/SiteAttachments/Demystifying-Convertible-Bonds-April-2024.pdf>.

Legrand. *€500 million fixed-rate bond issue maturing 2035. Press Release*, 12 March 2025. Available at: <https://www.legrandgroup.com>.

Saint-Gobain. *Sustainability-Linked Financing Framework*, July 2022. Available at: <https://www.saint-gobain.com>.

Saint-Gobain. *€1.5 billion triple-tranche bond issue including a Sustainability-Linked Bond (SLB). Press Release*, 2 August 2022. Available at: <https://www.saint-gobain.com>.

South Australian Government Financing Authority (SAFA). *Term Sheet – Tranche 4, 2.25% Notes maturing 15 August 2024. Official Document*. Available at: <https://www.safa.sa.gov.au>.

VINCI. *€400 million cash-settled synthetic convertible bonds due 2030. Press Release*, 11 February 2025. Available at: <https://www.vinci.com>.

## Intelligence Desk

Mistral AI. *Official Website*. Available at: <https://mistral.ai/fr>.

Federal Reserve Economic Data (FRED). *Long-Term Government Bond Yields – United States (10Y)*. <https://fred.stlouisfed.org/series/IRLTLT01USM156N>.